

System and Device Programming Projects

Device Programming Part (Prof. Quer)

Project Q1

Graph Neural Network Inference on GPUs

Project's summary

Graph Neural Networks (GNNs) extend deep learning models to graph-structured data, with applications in recommender systems, social network analysis, cybersecurity, bioinformatics, and industrial recommendation.

The goal of this project is to develop an inference engine for GNNs on multi-core CPUs (C++) and GPUs (CUDA), and to evaluate how different parallelization strategies (vertex parallel, edge parallel, message-passing batching, sparse versus dense representations) affect performance and scalability on large graphs.

Problem definition

Consider a directed or undirected graph $G = (V, E)$ with node features and, optionally, edge features. A typical GNN layer performs, for each node $v \in V$:

1. Aggregation of neighbors' features (sum, mean, or attention-based weighted combination).
2. Update of the node representation through a neural function (e.g., MLP).

The project requires:

- Implement one or two basic GNN architectures (e.g., GCN and/or GraphSAGE) for inference only, assuming pre-loaded weights.
- Design a sequential C/C++ implementation operating on efficient graph formats (CSR/CSC).
- Design a parallel CPU implementation (explicit threads, tasks, or OpenMP) exploiting parallelism over nodes, edges, or layers.
- Design one or more CUDA versions exploring different mappings (thread per node, per edge, per feature; use of shared memory; coalesced access on CSR structures).

Compare performance in terms of:

- Throughput (nodes/s or edges/s), memory footprint, and scalability with respect to the graph size (from tens of thousands up to millions of nodes).
- Feature vector size.
- Model depth (number of layers).

Implementation Details

Assume a single, large, static graph $G = (V, E)$, directed or undirected.

- Each vertex $v \in V$ has:
 - A unique integer id $v \in \{0, \dots, |V| - 1\}$.
 - A feature vector $x_v \in \mathbb{R}^F$ (node attributes).
 - Optionally, a label y_v (e.g., class id) used only for accuracy evaluation on a node-classification task.
- Each edge $(u, v) \in E$ has:

- **Optional** edge feature vector $e_{uv} \in \mathbb{R}^{F_e}$ (for GraphSAGE with edge features or attention mechanisms).

A GNN layer performs, for every node v :

1. Aggregation

- Collect neighbor representations $h_u^{(l)}$ for $u \in \mathcal{N}(v)$.
- Apply an aggregation operator, e.g.:
 - Sum: $m_v^{(l)} = \sum_{u \in \mathcal{N}(v)} h_u^{(l)}$.
 - Mean: $m_v^{(l)} = \frac{1}{|\mathcal{N}(v)|} \sum_{u \in \mathcal{N}(v)} h_u^{(l)}$.
 - Attention-weighted sum (optional extension).

2. Update

- Combine current node state and aggregated message through a fixed neural function:
 - GCN-style: $h_v^{(l+1)} = \sigma(W^{(l)} m_v^{(l)})$
 - GraphSAGE-style: $h_v^{(l+1)} = \sigma(W^{(l)} \cdot \text{concat}(h_v^{(l)}, m_v^{(l)}))$
- Weights $W^{(l)}$ and all layer parameters are preloaded and fixed (inference only).

The overall model is a stack of L such layers:

- Input: $h_v^{(0)} = x_v$.
- For $l = 0, \dots, L - 1$, compute $h_v^{(l+1)}$ for all v by applying the layer in parallel on the graph.
- Output:
 - Node classification: apply a final linear layer or softmax on $h_v^{(L)}$ to produce logits for each node, then (optionally) compute accuracy on a given split.
 - You can explicitly state "focus on full-batch inference on a single large graph, not mini-batches across many small graphs." [file:1]

The parallelization dimensions to explore are across nodes, edges, feature dimensions, and possibly across layers. Students should treat each GNN layer as one "graph-wide iteration" of message passing. For performance evaluation, each configuration (graph, feature size, depth, implementation) should be run multiple times to get stable timings.

Public datasets and formats

Large graphs with node features for GNNs are publicly available:

- Open Graph Benchmark (OGB) - large, realistic graphs for node and link prediction, with standardized loaders and splits.
 - Website: <https://ogb.stanford.edu>
 - Datasets such as ogbn-products, ogbn-papers100M, ogbn-arxiv provide:
 - A single large graph.
 - Node features and labels.
 - Standard tasks and splits.
 - The official PyTorch Geometric / DGL loaders expose the graph in CSR-like adjacency structures; students can export adjacency and features to your own CSR + dense-matrix format.
- Planetoid / citation datasets (Cora, CiteSeer, PubMed) - classic small-to-medium node-classification benchmarks with node features and labels.
 - Hosted in many repos; a canonical reference is the Planetoid data used by GCN and PyG.

- Even if distributed in preprocessed NumPy/SciPy form, they are fundamentally sparse adjacency matrices + dense node feature matrices, directly mappable to CSR + dense features.
 - They can serve as “small graphs” for functional debugging and as part of the required “public benchmarks” in your text.
- For synthetic graphs, students can generate:
 - Scale-free (Barabási-Albert), Erdos-Rényi random, and small-world graphs using standard libraries, then export to CSR.

Required background

The required background is covered by:

- Advanced programming courses (data structures, graphs, memory management).
- “System and Device Programming” course (concurrent programming, memory models).
- Basic CUDA programming (thread hierarchy, memory hierarchy, synchronization).

A short introduction to GNNs can be provided via selected survey/tutorial papers; students are not required to implement training, only the forward pass.

Working Environment

Students may work on a laptop or desktop, under Unix-like or Windows systems.

The implementation must be done in C/C++ with CUDA extensions for the GPU part.

The use of the following is encouraged:

- Lightweight utilities to load graph files (edge lists or custom CSR).
- Profiling tools (nvprof, Nsight Systems/Compute, perf) for performance analysis.

The project can naturally be extended into an MSc thesis by adding mini-batching across multiple graphs, graph partitioning and load-balancing techniques, or multi-GPU support.

Deliverables

All projects must be delivered including:

- C/C++ and CUDA source files.
- A README text file (plain ASCII) with instructions on how to compile and run the program, execution parameters, and dataset formats.
- A DOCUMENTATION file (Word, LaTeX, Markdown, etc.) containing:
 - main design choices (graph format, memory layout, CPU and GPU parallelization schemes, strategies for handling highly skewed degree distributions);
 - an experimental evaluation with tables and plots comparing:
- Sequential vs multi-core vs GPU implementations;
 - different parallelization strategies (vertex-centric vs edge-centric, with/without shared memory, with/without feature compression);
 - different graph and feature sizes.
 - experiments on both synthetic graphs (scale-free, random, small-world generators) and public graph benchmarks cited in the references.
- A set of OVERHEAD slides (PowerPoint or similar) for a 20-minute oral presentation of the projects.

Contact

Prof. Stefano Quer (stefano.quer@polito.it).

References

- Course material for "System and Device Programming", for general parallel programming concepts.
- Introductory tutorials and surveys on Graph Neural Networks and message-passing architectures.
- NVIDIA CUDA documentation (Programming Guide, Best Practices) for memory and parallelism optimizations on GPUs.
- Research papers on efficient GPU implementations of GNNs and on optimized sparse matrix operations.

Project Q2

Multi-Agent AI Framework for Industrial Fleet Management and Autonomous Troubleshooting

Background

AROL S.p.A. is a world leader in the design and production of advanced capping and filling machinery for bottles and jars, as well as capsule production. The company features high vertical integration: 95% of components and applications are developed in-house, ensuring total control over quality and production efficiency. With operational headquarters in Turin, other Italian cities, and key markets including the USA, Mexico, Brazil, China, and India, AROL provides a global presence serving various industrial sectors.

In recent years, AROL has undergone significant technological evolution, similar to that of the automotive sector. This shift has led to an increasing integration of advanced IT skills for centralized data management and machinery performance monitoring. AROL stands out for adopting advanced solutions across operating systems and system programming, web programming, modern user interfaces, machine learning, applied AI, and image recognition for quality control.

Project's Summary

As part of a broader Digital Business Transformation, AROL is developing a customer-dedicated platform to facilitate direct, immediate communication between the company and end users.

The goal is to provide customers with an advanced fleet management tool that lets them easily access all information about their machines: from the initial quote (including historical revisions) to orders, contractual documents, user manuals, and IoT data.

This platform serves as a digital archive collecting the "know-how" of the customer's machinery. Leveraging the vast amount of available data, AROL aims to integrate an agentic AI solution.

This AI will assist customers in retrieving information and providing operational support, such as technical troubleshooting.

By scanning a QR code on the machine, the end-user should quickly access the platform section dedicated to the user manual. Here, in addition to viewing the PDF, the user will be greeted by an AI Chatbot that guides them through the platform and, by invoking a series of AI Agents, provides all necessary support information.

The required activity involves developing the back-end and front-end of the chatbot, the AI agent architecture, and the relative orchestrator.

System Architecture

1. Frontend Interface: A responsive web dashboard (React/Angular/Vue) accessible via QR code, optimized for mobile industrial use.
2. Back-end Services: A microservices architecture (Node.js/Python) to manage user sessions and bridge the UI with the AI Orchestrator.
3. AI Orchestrator: A central "brain" (using frameworks like LangGraph or AutoGen) that interprets user intent and delegates tasks to specialized agents.

4. Specialized Agentic Layer:
 - o Doc-Agent: Uses RAG (Retrieval-Augmented Generation) to extract precise instructions from technical PDFs.
 - o Telemetry-Agent: Connects to IoT data streams to diagnose machine health status.
 - o Business Agent: Retrieves order history and maintenance contracts from the corporate database.

Objectives

The project aims to design and implement an intelligent ecosystem composed of a conversational interface (Chatbot) and a multi-agent AI system. The system must be capable of performing "reasoning" over heterogeneous data sources (PDF manuals, IoT telemetry, contractual history) to provide real-time expert assistance to plant operators.

Required Background

Knowledge of AI, Machine Learning, Neural Networks, Agents.

Working Environment

Students may work on a laptop or desktop, under Unix-like or Windows systems.
The project can be naturally extended into an MSc thesis.

Deliverables

All projects must be delivered including:

- Source files.
- A README text file (plain ASCII) with instructions on how to compile and run the program, execution parameters, and dataset formats.
- A DOCUMENTATION file (Word, LaTeX, Markdown, etc.) containing:
 - o main design choices (graph format, memory layout, CPU and GPU parallelization schemes, strategies for handling highly skewed degree distributions);
 - o an experimental evaluation with tables and plots comparing:
- A set of OVERHEAD slides (PowerPoint or similar) for a 20-minute oral presentation of the projects.

Contact

Prof. Stefano Quer (stefano.quer@polito.it).
AROL in-house staff

Additional Information

Additional details will be provided during the visit to Canelli on May 20th, 2026. It is recommended that at least one member of each group working on this activity attends the visit.

Project Q3

Multi-Agent System (MAS) for Industrial IoT Data Refinement and Analytics

Background

AROL S.p.A. is a world leader in the design and production of advanced capping and filling machinery for bottles and jars, as well as capsule production. The company features high vertical integration: 95% of components and applications are developed in-house, ensuring total control over quality and production efficiency. With operational headquarters in Turin, other Italian cities, and key markets including the USA, Mexico, Brazil, China, and India, AROL provides a global presence serving various industrial sectors.

In recent years, AROL has undergone significant technological evolution, similar to that of the automotive sector. This shift has led to an increasing integration of advanced IT skills for centralized data management and machinery performance monitoring. AROL stands out for adopting advanced solutions across operating systems and system programming, web programming, modern user interfaces, machine learning, applied AI, and image recognition for quality control.

Project's Summary

The objective of this project is to design and implement a **Multi-Agent System (MAS)** capable of processing raw data streams from industrial equipment (specifically, **Equatorque**, a modern capping machine). Students will develop a decentralized architecture in which autonomous agents collaborate to ingest, clean, and analyze high-frequency telemetry data stored locally (previously synchronized from the Cloud).

Objectives

The main objectives are the following:

- **Data Ingestion & Local Synchronization:** Implement a mechanism to pull raw datasets from the Cloud and manage a local data persistence layer.
- **Agent-Based Data Cleaning:** Develop specialized "Cleaning Agents" that apply logic to filter noise, specifically addressing the mismatch between polling frequency and the actual production cycle (slower update rate).
- **Deduplication & Logic Filtering:** Create rules to identify and remove redundant entries while maintaining the integrity of the capping process timestamps.
- **Advanced Data Analytics:** Implement "Analytic Agents" that perform higher-level reasoning on the cleaned data (e.g., performance KPIs, anomaly detection, or trend analysis).
- **Scalability & Autonomy:** Demonstrate how the agent-based approach handles increasing data volumes better than traditional monolithic scripts.

Required Background

To successfully complete this project, students should possess, or be willing to acquire, knowledge in:

- **Distributed Systems:** Understanding of agent communication protocols and coordination models.
- **Industrial IoT (IIoT):** Familiarity with data structures generated by PLC-controlled machinery (capping torque, angle, success/failure flags).
- **Programming Skills:** Proficiency in Python or Java.
- **Data Management:** Basic SQL/NoSQL knowledge for local storage and data manipulation (Pandas/NumPy).

Working Environment

Students may work on a laptop or desktop, under Unix-like or Windows systems.
The project can be naturally extended into an MSc thesis.

Deliverables

All projects must be delivered including:

- Source files.
- A README text file (plain ASCII) with instructions on how to compile and run the program, execution parameters, and dataset formats.
- A DOCUMENTATION file (Word, LaTeX, Markdown, etc.) containing:
 - main design choices (graph format, memory layout, CPU and GPU parallelization schemes, strategies for handling highly skewed degree distributions);
 - an experimental evaluation with tables and plots comparing:
- A set of OVERHEAD slides (PowerPoint or similar) for a 20-minute oral presentation of the projects.

Contact

Prof. Stefano Quer (stefano.quer@polito.it).
AROL in-house staff

Additional Information

Additional details will be provided during the visit to Canelli on May 20th, 2026. It is recommended that at least one member of each group working on this activity attends the visit.